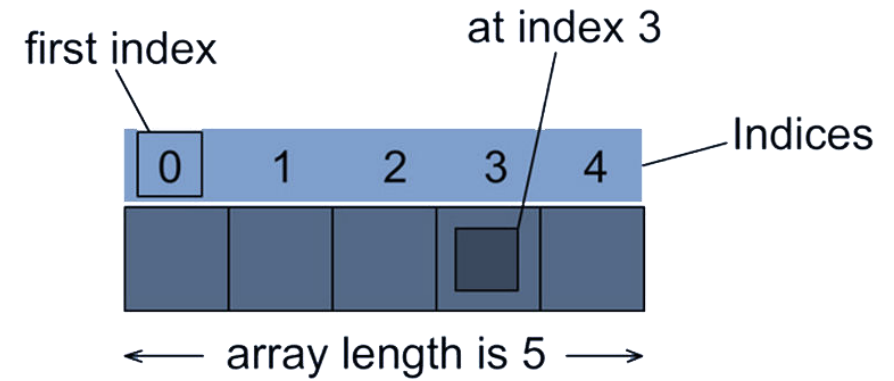


Linear Data Structure Array



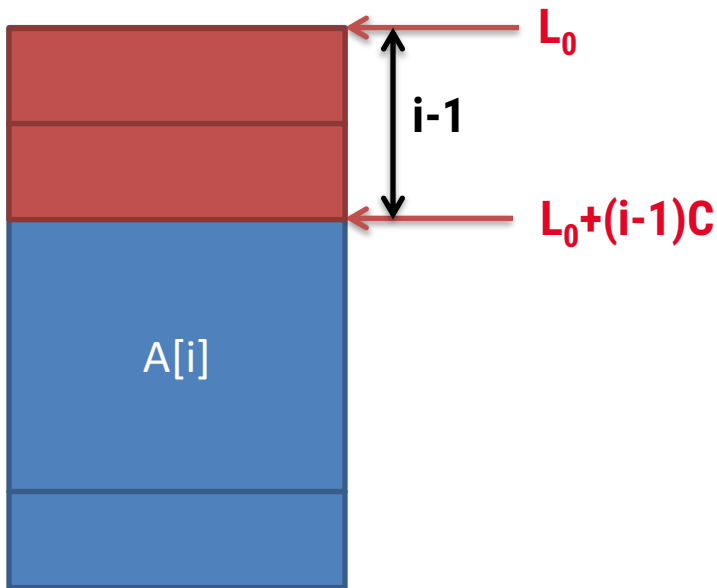


Outline

- Array
 - Representation of arrays
 - One dimensional array
 - Two dimensional array
- Applications of arrays
 - Symbol Manipulation (matrix representation of polynomial equation)
 - Sparse matrix
- Sparse matrix and its representation

One Dimensional Array

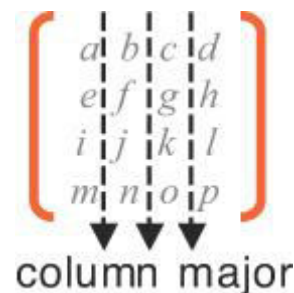
- ▶ Simplest data structure that makes use of computed address to locate its elements is the one-dimensional array or vector.
- ▶ Number of memory locations is sequentially allocated to the array.
- ▶ A array size is fixed and therefore requires a fixed number of memory locations.
- ▶ Array A with subscript lower bound of “one” is represented as below....



- L_0 is the address of the first word allocated to the first element of array A
- C words is size of each element or node
- The address of element A_i is
$$Loc(A_i) = L_0 + (C*(i-1))$$
- Let's consider the more general case of a array A with lower bound for it's subscript is given by some variable b.
- The address of element A_i is
$$Loc(A_i) = L_0 + (C*(i-b))$$

Two Dimensional Array

- ▶ Two dimensional arrays are also called **table** or **matrix**
- ▶ Two dimensional arrays have two subscripts
- ▶ **Column major order matrix:** Two dimensional array in which elements are stored column by column is called as column major matrix
- ▶ Two dimensional array consisting of **two rows** and **four columns** is stored sequentially by columns: $A[1,1]$, $A[2,1]$, $A[1,2]$, $A[2,2]$, $A[1,3]$, $A[2,3]$, $A[1,4]$, $A[2,4]$



	Col-1	Col-2	Col-3	Col-4
Row 1	[1,1]	[1,2]	[1,3]	[1,4]
Row 2	[2,1]	[2,2]	[2,3]	[2,4]

Column major order matrix

	Col-1	Col-2	Col-3	Col-4
Row 1	[1,1]	[1,2]	[1,3]	[1,4]
Row 2	[2,1]	[2,2]	[2,3]	[2,4]

- ▶ The address of element A [i , j] can be obtained by expression

$$\text{Loc (A [i , j])} = L_0 + (j-1)*2 + (i-1)$$

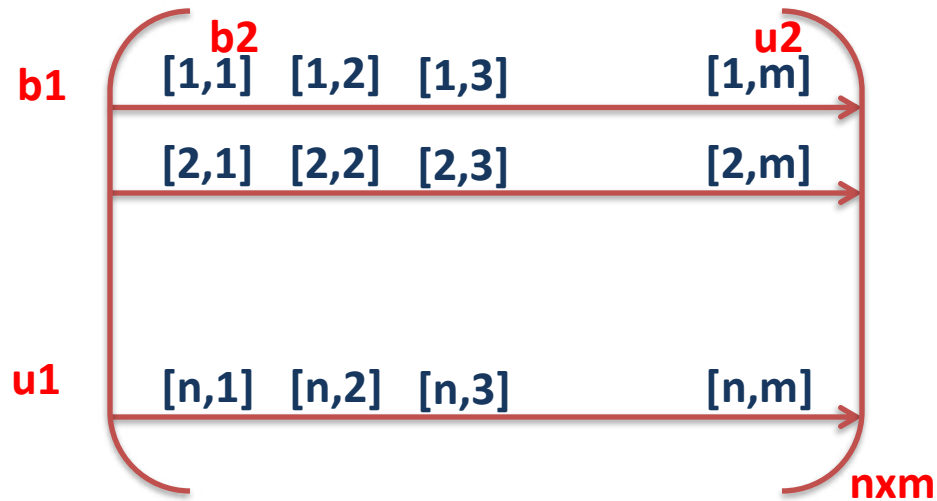
$$\text{Loc (A [2, 3])} = L_0 + (3-1)*2 + (2-1) = \mathbf{L_0 + 5}$$

- ▶ In general for two dimensional array consisting of **n rows** and **m columns** the address element A [i , j] is given by

$$\text{Loc (A [i , j])} = L_0 + (j-1)*n + (i - 1)$$

Row major order matrix

- **Row major order matrix:** Two dimensional array in which elements are stored row by row is called as row major matrix



n = no of rows, **m** = no of columns

b1 = lower bound subscript of row

u1 = upper bound subscript of row

n = $u1 - b1 + 1$

b2 = lower bound subscript of column

u2 = upper bound subscript of column

m = $u2 - b2 + 1$

- The address element $A[i, j]$ is given by

$$\text{Loc}(A[i, j]) = L_0 + (i-1)*m + (j - 1)$$

- The address element $A[i, j]$ is given by

$$\text{Loc}(A[i, j]) = L_0 + (i-b1)*(u2-b2+1) + (j - b2)$$

Applications of Array

1. Symbol Manipulation (matrix representation of polynomial equation)
2. Sparse Matrix

► Matrix representation of polynomial equation

- ➡ We can use array for different kind of operations in polynomial equation such as addition, subtraction, division, differentiation etc...
- ➡ We are interested in finding suitable representation for polynomial so that different operations like addition, subtraction etc... can be performed in efficient manner.
- ➡ Array can be used to represent Polynomial equation.

Representation of Polynomial equation

	Y	Y ²	Y ³	Y ⁴
X	XY	XY ²	XY ³	XY ⁴
X ²	X ³ Y	X ² Y ²	X ² Y ³	X ² Y ⁴
X ³	X ³ Y	X ³ Y ²	X ³ Y ³	X ³ Y ⁴
X ⁴	X ⁴ Y	X ⁴ Y ²	X ⁴ Y ³	X ⁴ Y ⁴

	Y	Y ²	Y ³	Y ⁴
X	XY	XY ²	XY ³	XY ⁴
X ²	0	1	1	X ² Y ⁴
X ³	-1	3	0	X ³ Y ⁴
X ⁴	1	0	0	X ⁴ Y ⁴

$2X^2 + 5XY + Y^2$

		Y	Y ²	Y ³	Y ⁴
	0	0	1	0	0
X	0	5	0	0	0
X ²	2	0	0	0	0
X ³	0	0	0	0	0
X ⁴	0	0	0	0	0

$X^2 + 3XY + Y^2 + Y - X$

		Y	Y ²	Y ³	Y ⁴
	0	1	1	0	0
X	-1	3	0	0	0
X ²	1	0	0	0	0
X ³	0	0	0	0	0
X ⁴	0	0	0	0	0

Sparse matrix

- ▶ An $m \times n$ matrix is said to be **sparse** if “many” of its elements are zero.
- ▶ A matrix that is not sparse is called a **dense matrix**.
- ▶ We can devise a simple representation scheme whose space requirement equals the size of the non-zero elements.

	Column - 1	Column - 2	Column - 3	Column - 4	Column - 5	Column - 6	Column - 7	Column - 8
Row - 1	0	0	0	2	0	0	1	0
Row - 2	0	6	0	0	7	0	0	3
Row - 3	0	0	0	9	0	8	0	0
Row - 4	0	4	5	0	0	0	0	0

4x8

Terms	0	1	2	3	4	5	6	7	8
Row	1	1	2	2	2	3	3	4	4
Column	4	7	2	5	8	4	6	2	3
Value	2	1	6	7	3	9	8	4	5

Linear Representation of given matrix

Sparse matrix Cont...

- ▶ To construct matrix structure from liner representation we need to record.
 - ➔ Original row and columns of each non zero entries.
 - ➔ Number of rows and columns in the matrix.
- ▶ So each element of the array into which the sparse matrix is mapped need to have three fields:
row, column and value

Sparse matrix Cont...

A =

	1	2	3	4	5	6	7
1	0	0	6	0	9	0	0
2	2	0	0	7	8	0	4
3	10	0	0	0	0	0	0
4	0	0	12	0	0	0	0
5	0	0	0	0	0	0	0
6	0	0	0	3	0	0	5

6x7

Memory Space required to store
6x7 matrix

$$42 \times 2 = 84 \text{ bytes}$$

Memory Space required to store
Linear Representation

$$30 \times 2 = 60 \text{ bytes}$$

Linear representation of Matrix

Row	Column	A
1	3	6
1	5	9
2	1	2
2	4	7
2	5	8
2	7	4
3	1	10
4	3	12
6	4	3
6	7	5

$$\text{Space Saved} = 84 - 60 = 24 \text{ bytes}$$

Sparse matrix Cont...

Linear Representation of Matrix

Row	Column	A
1	3	6
1	5	9
2	1	2
2	4	7
2	5	8
2	7	4
3	1	10
4	3	12
6	4	3
6	7	5

Linear Representation of Matrix

Row	Column	A
1	3	6
2	5	9
3	1	2
4	4	7
5	5	8
6	7	4
7	1	10
8	3	12
9	4	3
10	7	5

Memory Space required to store Liner Representation = $26 \times 2 = 42$ bytes

Different ways to initialize an Array

▶ Method 1: Initialize an array using an Initializer List

- ➔ `int arr[5] = {1, 2, 3, 4, 5};`
- ➔ `int arr[5] = {0};` all five element are zero
- ➔ `int arr[3][3] = {1,2,3,4,5,6,7,8,9};`

▶ Method 2: Initialize an array in C using a for loop

- ➔ `int arr[5];`
- ➔ `for (int i=0; i<5; i++)`
- ➔ `arr[i] = i;` or used `scanf` statement

▶ Method 3: Using Designated Initializers (For gcc compiler only)

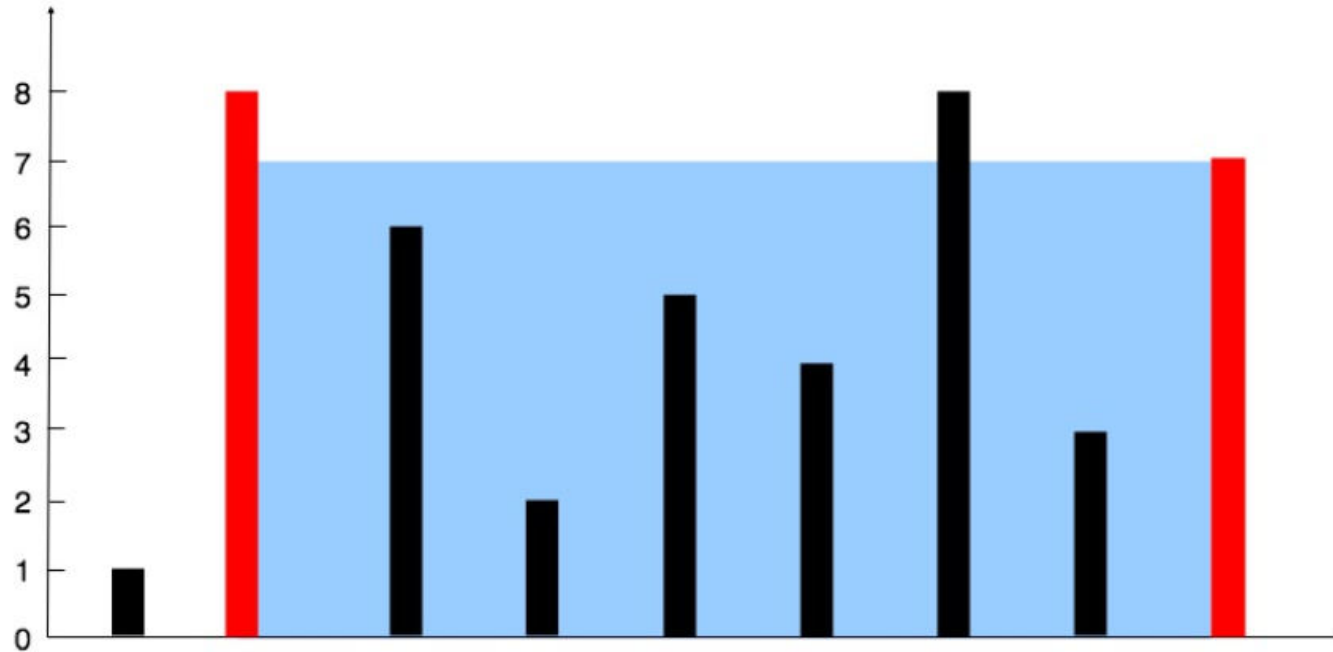
- ➔ `int arr[9] = { [0 ... 8] = 10 };`
- ➔ `int arr[9] = { 0, [1 ... 7] = 10, 0 };`

Problem 1

- ▶ Sort the given array in $O(n)$ time
- ▶ $A = [0, 1, 1, 1, 0, 0, 0, 1, 1, 0]$

Problem 3

- ▶ You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are $(i, 0)$ and $(i, \text{height}[i])$.
- ▶ Find two lines that together with the x-axis form a container, such that the container contains the most water.



Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

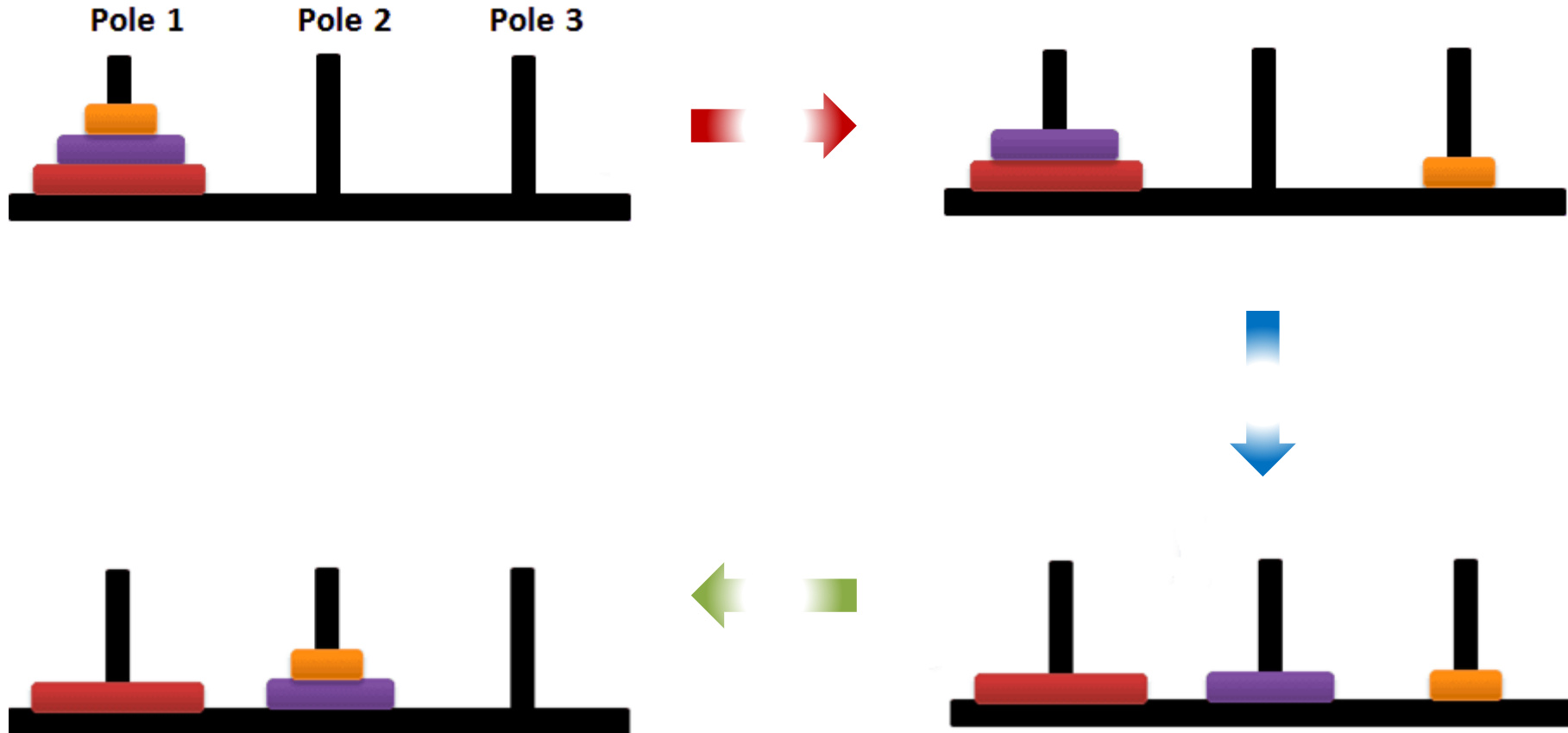
Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Puzzle

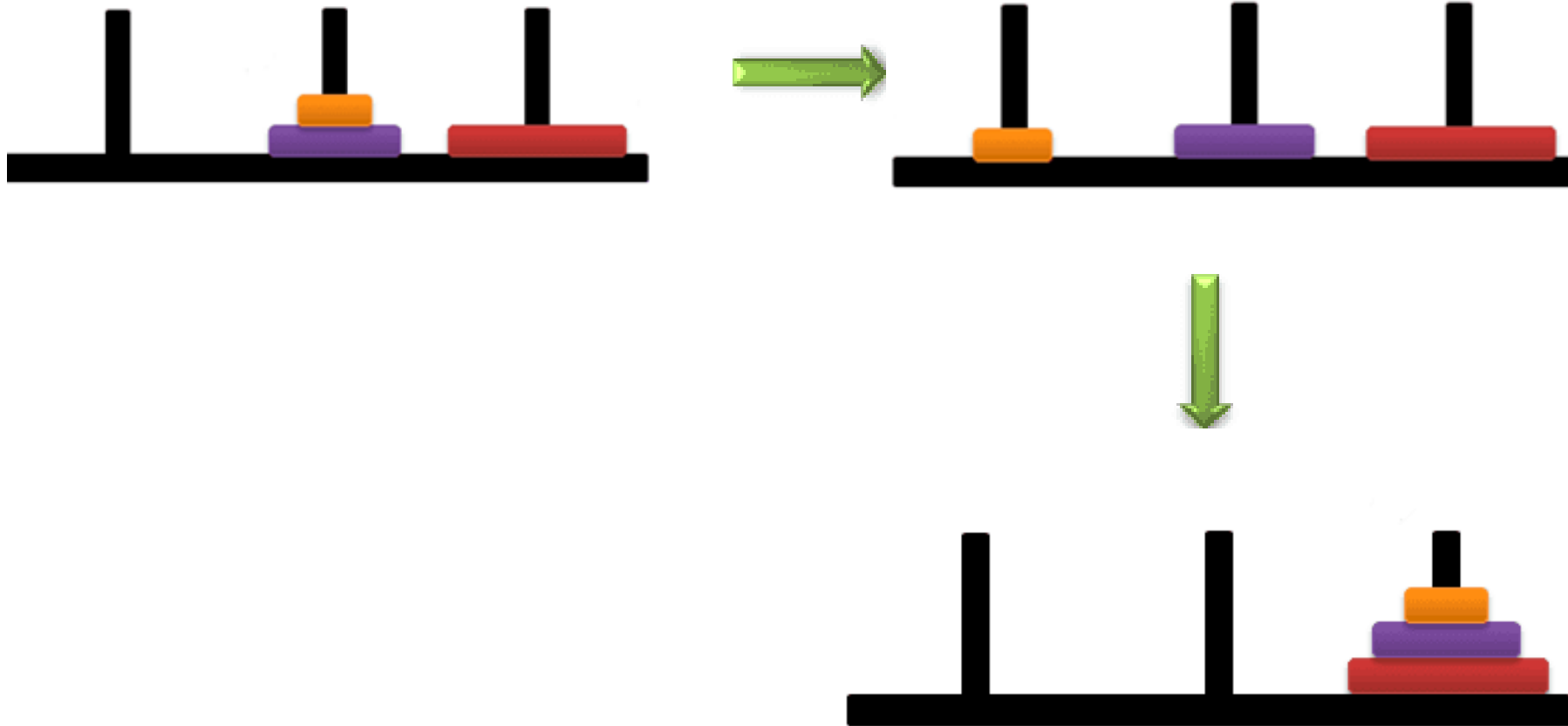
- ▶ 1. There are two gates, one to hell and the other to heaven. Two gatekeepers, one for each gate. One of them always speaks the truth and the other always lies but you don't know which one guards which gate. You are allowed only one question and you need to find out the gate to heaven. What is the question?
- ▶ 2. There are 3 jars, namely, A, B, C. All of them are mislabeled. Following are the labels of each of the jars: **A:Candies B:Sweets C:Candies and Sweets** (mixed in a random proportion) You can put your hand in a jar and pick only one eatable at a time. Tell the minimum number of eatable(s) that has/have to be picked in order to label the jars correctly.

- ▶ 3. How do we measure forty-five minutes using two identical wires, each of which takes an hour to burn? We have matchsticks with us. The wires burn non-uniformly. So, for example, the two halves of wire might burn in 10 minutes and 50 minutes respectively.
- ▶ 4. There are 4 persons (A, B, C and D) who want to cross a bridge in night.
 - ➔ a. A takes 1 minute to cross the bridge.
 - ➔ b. B takes 2 minutes to cross the bridge.
 - ➔ c. C takes 5 minutes to cross the bridge.
 - ➔ d. D takes 8 minutes to cross the bridge.
- ▶ There is **only one torch** with them and the bridge cannot be crossed without the torch. There cannot be more than two persons on the bridge at any time, and when two people cross the bridge together, they must move at the slower person's pace. How much time they will take to cross the bridge.

Analysis: Tower of Hanoi



Analysis: Tower of Hanoi



**Thank
You**